

Verifying Hyperproperties with TLA

Authors: Leslie Lamport, Fred B. Schneider

Presented by: Sebastian David Albu

Formal Specification and Design of Software, WS 2024

Definitions and Overview

State

- A state describes current assignment of variables.
- A system is usually described by a state machine in which state transitions generate events.
- We call a sequence of states a *behavior*.
- We call a pair of consecutive states in a behavior a *step*.

State Machines

- Start: An initial predicate $\mathcal{I}$
- Example:
 - We can describe an hour-minute clock by a state containing variables `hr` and `min` that represent the hour and minute, respectively. Here, the initial state is 12:00.

■

$$\mathcal{I}_{hm} \triangleq (\text{min} = 0) \wedge (\text{hr} = 12)$$

State Machines

- Next state predicate $\mathcal{N}$
- There needs to be a finite set

State Machines

■ Example:

■

$$\begin{aligned} \mathcal{N}_{hm} \triangleq & (\text{min}' = \text{min} + 1 \pmod{60}) \quad \wedge \\ & (\text{hr}' = \text{IF } (\text{min} = 59) \\ & \quad \text{THEN IF } (\text{hr} = 12) \\ & \quad \quad \text{THEN } 1 \\ & \quad \quad \text{ELSE } \text{hr} + 1 \\ & \quad \text{ELSE hr}) \end{aligned}$$

Properties

- Since we represent executions as behaviors, a property is a predicate on behaviors.
- For a single behavior b satisfying a predicate P :

$$b \models P \iff P \text{ is true for } b$$

- For a set of behaviors S satisfying P :

$$S \models P \iff \forall b \in S : b \models P$$

- Predicate on behaviors and set of states satisfying the predicate:

$$S \models P \iff S \subseteq P \iff \models (S \Rightarrow P)$$

Hyperproperties

- Properties cannot directly describe certain security conditions, so they were generalized to hyperproperties.
- A hyperproperty is a predicate on sets of behaviors rather than on a single behavior, making it a predicate on properties.
- An example is the hyperproperty $\mathcal{H}$ where, for a property $\mathcal{P}$, we define $\mathcal{H}(\mathcal{P})$ to be true iff:
 - Any two terminating behaviors satisfying $\mathcal{P}$ that have different initial values of x have different terminal values of y .

Hyperproperties and Quantification

- A formula is *finitary* if it does not depend on P , a predicate on sets of behaviors.

$$\forall \exists b_1 \in P : \dots \forall \exists b_k \in P : J(b_1, \dots, b_k)$$

$$\forall b_1 \in P : \dots \forall b_j \in P : K(b_1, \dots, b_j) \Rightarrow \exists b_{j+1} \in P : \dots \exists b_k \in P : L(b_1, \dots, b_k)$$

$$\begin{aligned} \forall b_1, \dots, b_j : P(b_1) \wedge \dots \wedge P(b_j) \wedge K(b_1, \dots, b_j) \Rightarrow \\ \exists b_{j+1}, \dots, b_k : P(b_{j+1}) \wedge \dots \wedge P(b_k) \wedge L(b_1, \dots, b_k) \end{aligned}$$

- $b_1, \dots, b_k$: Individual behaviors in the set P .
- P : A predicate defining conditions on sets of behaviors.
- J, K, L : Logical formulas relating behaviors, defining hyperproperties.
- **Prenex Normal Form (PNF)**: A formula is in PNF if all quantifiers (e.g., $\forall, \exists$) precede a quantifier-free expression.

Temporal Logic & Operators

Temporal Operator $\Box$ (Always/Henceforth)

- The temporal operator $\Box$ (read "always" or "henceforth") means:

$$b \models \Box F \iff F \text{ is true for all suffixes of } b.$$

- Example: $b \models \Box(x > y)$ holds iff $x > y$ is true in every state of behavior b .
- In temporal logic, variables can change values across states of a behavior, similar to variables in programming.

Temporal Quantifier $\exists$

- The existential quantifier $\exists x : F$ in temporal logic asserts:

$b \models \exists x : F$ iff there exists a behavior b'

identical to b except that x 's values may differ across states, such that $b' \models F$.

- Unlike predicate logic's $\exists x : F$, temporal $\exists x : F$ is true if a sequence of values for x makes F true in each state.
- Temporal $\exists$ follows the same rules as $\exists$ in predicate logic.

Behavior in RTLA

- A behavior b is a possible behavior of the state machine described by

$$b \models \mathcal{I} \wedge \Box \mathcal{N}$$

iff:

- $b \models \mathcal{I}$ asserts that the first state of b satisfies $\mathcal{I}$.
- $b \models \Box \mathcal{N}$ asserts that the first step of every suffix of b satisfies $\mathcal{N}$ (since every step of b is the first step of a suffix of b).

Example (Using y as a hidden variable)

- True iff $x \neq 1$ unless x was 42 before

■

$$\exists y : (y = (x = 42 \wedge \Box((\neg y \Rightarrow (x' \neq 1)) \wedge (y' = (y \vee (x = 42)))))$$

Generalized noninterference (GNI) in RTLA

- Hyperproperty that was proposed as a security condition for systems.
- Stated in a model where an execution is described as a sequence of events rather than a sequence of states. Events are classified as public, visible to all observers, or secret, visible only to privileged observers.

■

$$\forall b_1, b_2 : P(b_1) \wedge P(b_2) \Rightarrow \exists b_3 : P(b_3) \wedge L(b_1, b_2, b_3)$$

- $P(b_i)$ asserts b_i is possible behavior of system
- $L(b_1, b_2, b_3)$ asserts GNI condition $\Rightarrow$ public b_3 always same as b_1 , secret b_3 always same as b_2 .

Understanding $L(b_1, b_2, b_3)$ in GNI

- $L(b_1, b_2, b_3)$ ensures consistency for public and secret behavior:
 - Public events: b_3 matches b_1 .
 - Secret events: b_3 matches b_2 .
- Example:
 - b_1 : A public execution with events $\{e_{pub1}, e_{pub2}\}$.
 - b_2 : A secret execution with events $\{e_{sec1}, e_{sec2}\}$.
 - b_3 : Combines $\{e_{pub1}, e_{pub2}\}$ from b_1 and $\{e_{sec1}, e_{sec2}\}$ from b_2 .
- L guarantees no mixing of public and secret behavior between observers.

Difference Between GNI and Regular NI

■ Noninterference (NI):

- Single execution trace.
- Secret actions have no effect on public outputs.
- Focuses solely on *state transitions*.

■ Generalized Noninterference (GNI):

- Multiple execution traces (b_1, b_2, b_3).
- Captures hyperproperties (relations between executions).
- Accounts for both public and secret visibility of *events*, not just *states*.

■ Key Difference:

- NI: Isolation of secrets in a single trace.
- GNI: Consistency of secrets and public behavior across multiple traces.

Hyperproperties as Properties

- Rewrite the Hyperproperty definition below to temporal logic



$$\begin{aligned} &\forall b_1, \dots, b_j : P(b_1) \wedge \dots \wedge P(b_j) \wedge K(b_1, \dots, b_j) \Rightarrow \\ &\exists b_{j+1}, \dots, b_k : P(b_{j+1}) \wedge \dots \wedge P(b_k) \wedge L(b_1, \dots, b_k) \end{aligned}$$

- Steps:

- **Rewrite Formula:** Transform multiple behaviors into a single behavior formula.
- **Variable Substitution:** Encode each behavior b_i with unique variables $v_1, \dots, v_n = \mathbf{x}$.
- **Simplified Notation:** Use $\tilde{P}(x)$ to represent the modified formula for each behavior's variables.
- **Predicate Translation:** Replace $P(b_i)$ with $\tilde{P}(x_i)$ in the formula.
- **Temporal Logic Representation:** Final formula becomes a temporal predicate on a single behavior b combining P , K , and L .

Hyperproperties as Properties

- For convenience now P satisfies temporal formula $\tilde{P}$ and predicate on behaviors it represents.

■

$$\models P(\mathbf{x}_1) \wedge \dots \wedge P(\mathbf{x}_j) \wedge K(\mathbf{x}_1, \dots, \mathbf{x}_j) \Rightarrow \\ \exists \mathbf{x}_{j+1}, \dots, \mathbf{x}_k : P(\mathbf{x}_{j+1}) \wedge \dots \wedge P(\mathbf{x}_k) \wedge L(\mathbf{x}_1, \dots, \mathbf{x}_k)$$

- $L(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3)$ asserts GNI condition $\Rightarrow$ public $\mathbf{x}_3$ always same as $\mathbf{x}_1$, secret $\mathbf{x}_3$ always same as $\mathbf{x}_2$.

■

$$L(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) \triangleq \Box(public(\mathbf{x}_1) = public(\mathbf{x}_3) \wedge secret(\mathbf{x}_1) = secret(\mathbf{x}_2))$$

From RTL to TLA: Stuttering

Stuttering Insensitivity in State Machines

■ Clock Example:

- S_{hm} : Describes an hour-minute clock, with only `hr` and `min` variables.
- S_{hms} : Extends S_{hm} to an hour-minute-second clock, adding the variable `scd`.

■ Stuttering Issue:

- $S_{hms} \models S_{hm}$ is invalid because S_{hms} allows 59 “stuttering” steps in which `scd` changes, but `hr` and `min` remain unchanged.
- This illustrates that S_{hm} only describes a clock within a minimal “universe” of `hr` and `min`, not accounting for `scd`.

- **Conclusion:** For a universe with more variables (e.g., `scd`), the description must permit stuttering steps, ensuring it holds in larger systems.

Stuttering Insensitivity (SI)

- Formula is still satisfied after adding/removing unchanged steps.
 - To obtain an SI RTLA formula that describes an hour-minute clock and does not constrain the rest of the universe, we redefine S_{hm} :

-

$$S_{hm} \triangleq \mathcal{I}_{hm} \wedge \Box(\mathcal{N}_{hm} \vee (\langle hr, min \rangle' = \langle hr, min \rangle))$$

- Same formula now in TLA:
 - Obtained by modifying RTLA s.t. every syntactically correct TLA formula is SI

-

$$S_{hm} \triangleq \mathcal{I}_{hm} \wedge \Box[\mathcal{N}_{hm}]_{\langle hr, min \rangle}$$

Stuttering Equivalent (SE)

- Removing repeated values produces identical sequences.
- Behaviors are SE if they match after ignoring stuttering steps (repeated states).
- SE allows systems to abstract away irrelevant state changes.

GNI and Combining Executions

■ Behavior Combination for GNI:

- Construct b_3 by combining executions of $\hat{b}_1$ and $\hat{b}_2$.
- Define $P(\hat{\mathbf{x}}_1)$ so b satisfies $P(\mathbf{x}_1)$ if values b assigns to $\mathbf{x}_1$ represent a behavior where P holds.

■ Defining Equivalent Executions:

- $b|_{\hat{\mathbf{x}}_1} = b|_{\mathbf{x}_1}$ implies equivalent behavior assignments.
- Replace $\text{public}(\mathbf{x}_3) = \text{public}(\mathbf{x}_1)$ with $\text{public}(\mathbf{x}_3) = \text{public}(\hat{\mathbf{x}}_1)$, allowing different $\hat{\mathbf{x}}_1, \hat{\mathbf{x}}_2$ values satisfying GNI. (Same for secret)

Temporal Operator $\sim$ for Stuttering Insensitivity (SI)

- Temporal operator $\sim$ definition:
 - $b \models (f \sim g)$ iff $b|_f$ is SE to $b|_g$
- Formula F is SI iff:
 - $\models (\mathbf{x} \sim \mathbf{y}) \Rightarrow (F(\mathbf{x}) = F(\mathbf{y}))$
- **Example: Generalized Non-Interference (GNI)**
 - $\models P(\mathbf{x}_1) \wedge P(\mathbf{x}_2) \Rightarrow \exists \hat{\mathbf{x}}_1, \hat{\mathbf{x}}_2, \mathbf{x}_3 : (\hat{\mathbf{x}}_1 \sim \mathbf{x}_1) \wedge (\hat{\mathbf{x}}_2 \sim \mathbf{x}_2) \wedge P(\mathbf{x}_3) \wedge L(\hat{\mathbf{x}}_1, \hat{\mathbf{x}}_2, \mathbf{x}_3)$
 - where $L(\hat{\mathbf{x}}_1, \hat{\mathbf{x}}_2, \mathbf{x}_3) \triangleq \Box(\text{public}(\mathbf{x}_3) = \text{public}(\hat{\mathbf{x}}_1) \wedge \text{secret}(\mathbf{x}_3) = \text{secret}(\hat{\mathbf{x}}_2))$

Temporal Operator $\stackrel{\#}{=}$ for SI

- Defines equality in action steps: $b \models A \stackrel{\#}{=} B$ if b has the same count of A and B steps.
- SI limitation: $\stackrel{\#}{=}$ is not SI if A or B can occur without changing variables.
- Solution: Use actions of the form $\langle C \rangle_v$, which only counts C steps where variables in v change. $\Rightarrow$ SI: $C \wedge (v' \neq v) \iff \langle C \rangle_v$
 - GNI example:

$$\models P(\mathbf{x}_1) \wedge P(\mathbf{x}_2) \Rightarrow (\langle Pub(\mathbf{x}_1) \rangle_{\langle \mathbf{x}_1 \rangle} \stackrel{\#}{=} \langle Pub(\mathbf{x}_2) \rangle_{\langle \mathbf{x}_2 \rangle})$$

Further Terminology and Examples of Hyperproperties

Preservation Under Refinement

- **Refinement Definition:** A system S refines P if S implies P , meaning S 's behaviors are a subset of P 's.
- **Hyperproperty Preservation:** A hyperproperty is preserved under refinement if, for any refined S , the hyperproperty remains satisfied.
- **Same vs. Different Variables:** When S and P share the same variables, refinement simply ensures that S 's behaviors are consistent with P 's specification (e.g., both use the same data types or state variables). However, when S and P use different variables (e.g., pixels in S vs. characters in P), refinement requires a mapping or correspondence between these variables to define how S satisfies P .
- **Examples:** Refinement can model different abstractions, like characters on a screen or packet-switching in networks.

Types of Hyperproperties

- **Generalized Non-Interference (GNI)**
 - Ensures that secret data does not affect public outputs across executions.
- **Observational Determinism (OD)**
 - Guarantees that identical inputs lead to identical observable behaviors.
- **Possible Non-Interference (PNI)**
 - Allows selective influence of secret data on public outputs under certain conditions.
- **Perfect Security Property (PSP)**
 - Requires complete non-leakage of secret information; problematic as it is nearly impossible to implement in practice.

Conclusion and Future Applications

Conclusion

- Hyperproperties extend traditional properties by describing relationships across multiple executions of a system, capturing essential security and privacy requirements.
- Verification of hyperproperties allows for checking complex system behaviors like information flow, ensuring non-interference and preventing unintended data leaks.
- Leveraging Temporal Logic of Actions (TLA) with self-composition provides a rigorous approach to verifying these properties in system designs.

Significance and Future Applications

- Hyperproperties support designing systems that are resilient to vulnerabilities by enabling fine-grained control over security and correctness.
- Preservation under refinement ensures that hyperproperties remain valid in lower-level implementations, linking abstract models to real-world applications.
- Future work involves extending verification techniques for more complex systems, such as distributed or probabilistic systems, to enhance robustness.

Questions?

Verifying Hyperproperties with TLA

Authors: Leslie Lamport, Fred B. Schneider

Presented by: Sebastian David Albu

Formal Specification and Design of Software, WS 2024